# BrandGuard

## Round 2 build specification: features, workflow, tech stack, hosting and build prompts

| | |
|---|---|
| Event | Codex 48 (Inkloom AI problem statement: brand-governance platform) |
| Round | Round 2: The Build. Opens Mon 26 Oct 10:00 AM IST, closes Wed 28 Oct 10:00 AM IST (hard deadline) |
| Team | BrandGuard (solo). Team leader and builder: Krrish Yaduka |
| Stack | React + Three.js frontend, Node.js/Express backend, MongoDB Atlas, Gemini (NVIDIA as fallback), Render hosting |
| Built with | Claude Code, guided by this document |

**How to use this document.** Sections 1 to 14 define what to build and why. Section 15 gives ready-to-paste prompts for Claude Code, one per build phase. Section 9 contains the exact demo data so the app works the moment a judge opens it.

### Contents


0. Round 2 at a glance
1. Problem statement
2. Proposed solution
3. What a judge must be able to do
4. Features and acceptance tests
5. Step-by-step product workflow
6. Review engine design
7. Technology stack
8. Architecture, data model and API
9. Demo data (seed)
10. Hosting and deployment
11. Responsible AI and security
12. Screens and the 3D plan
13. 48-hour build plan
14. Submission pack (features, changes, Round 3)
15. Claude Code prompts and repo structure
16. Risks and honest notes

# 0. Round 2 at a glance

The round is scored on one thing: a judge opens your link and uses the **core path** of the product without any help. Content goes in, it is checked against approved guidelines, terms, facts or assets, issues come back explained, and a human decides. A login screen, a settings page or an empty dashboard does not count.

| Submit (team leader only, via the link posted in the official group) | Notes |
|---|---|
| Working project link | Must open for anyone with no sign-in. Test in a private window |
| Source-code repository link | Public repository. No secrets committed |
| Updated project PDF | Section 14 of this document gives the content |
| List of completed features | Only list what a judge can actually use from the link |
| Technology and implementation details | Section 7 and 8 |
| Changes made to the original plan | Section 14 table. Be honest |
| Three major improvements for Round 3 | Must be real additions, not design tweaks |

**Hard rules:** deadline is Wed 28 Oct, 10:00 AM IST and late submissions are not scored. Only the leader submits. Watch the official group for the submission link; it is not emailed. Teams lost their place for links that needed a sign-in.

### How Round 2 is scored (100 points)

| Criterion | Pts | What wins it | Where in this spec |
|---|---|---|---|
| Core workflow runs end to end | 30 | The seven-step path works on the live link with no help, including reviewer approval and the reverse alert | Sections 3, 5. F2 to F12. Phase 8 browser test |
| Technical depth and reliability | 25 | Citation validator, verify-then-suggest loop, fact scanning, retrieval, AI fallback, engine tests, loading and error states, bad input never crashes the app | Sections 6, 11. F5, F7, F12, F18 |
| Progress against the plan | 20 | Judges compare against your Round 1 plan. The differentiators must visibly exist, and every change must be explained | Section 14 progress tracker |
| The three improvements you named | 15 | Specific, genuinely new, believable for 48 hours, with groundwork visible in Round 2 | Section 14 |
| Usability so far | 10 | Example buttons, clear labels, fast results, role switcher, no dead screens. 3D is a small part of this | Sections 3, 12 |

**55 of 100 points** depend on the core path working and on engineering quality. Only 10 points are for usability, so 3D animation is worth very little. Spend your time on reliability and the plan differentiators first.

# 1. Problem statement

Growing organisations publish content through many employees, agencies and AI tools, which makes it hard to keep a consistent and trustworthy brand. Incorrect claims, outdated information, inconsistent language and off-brand visuals damage credibility before anyone notices. The brief asks for a platform that reviews content **before publication** against approved guidelines, facts and assets, explains each issue, suggests corrections that keep the creator's purpose, and supports review and approval by humans.

### Who experiences it and how

| User | Pain today |
|---|---|
| Creator (employee, agency, freelancer, AI-tool user) | Does not know the rules, gets vague rejections, reworks repeatedly |
| Brand or marketing manager | Becomes the bottleneck; reviews everything by hand and still misses errors |
| Legal or compliance reviewer | Flooded with low-risk items; cannot focus on risky claims |
| Admin / brand owner | Facts go stale silently; no idea which guidelines are unclear |

### Three root causes we target

- **Facts expire silently.** An offer ends or a price changes but drafts and scheduled posts keep the old version.
- **Guidelines are ambiguous.** People read a rule differently, so it is broken constantly, and no tool says the rule is the problem.
- **Rejection without help.** Tools say no but do not preserve what the creator meant to say.

## 2. Proposed solution

**BrandGuard** is a pre-publication reviewer that checks content against the organisation's own approved material, explains each finding with the exact source it relies on, and suggests fixes it has already re-checked. Humans make every final decision.

| Idea | What it means in the product |
|---|---|
| **Citation or human** | Every finding cites a real rule ID or approved fact ID. If none exists the finding says 'no rule found, human decision needed' and goes to a reviewer. The model is never trusted to invent a rule |
| **Living facts** | Approved facts have an owner and an expiry date. When a fact changes or expires, BrandGuard scans every draft and approved item and alerts the owner (reverse alerts) |
| **Verified suggestions** | A suggested fix is shown only after the full checker is re-run on the fixed text and the original meaning is shown to be preserved |
| **Rules tested, not just enforced** | Rules that are often disputed, or that fail their own test cases, get a clarity score so owners can reword them |
| **Prevention at the source** | A simple API lets AI writing tools fetch the rules and self-check before anyone reviews (MCP version in Round 3) |

## 3. What a judge must be able to do

Design the whole app around this seven-step path. If any step breaks, nothing else matters.

**1.** Open the link. No login. A short banner says what BrandGuard does and offers 'Try an example'.

**2.** Click an example (or paste text, optionally upload an image) and press **Review content**.

**3.** See the text with highlighted problems and a list of findings: severity, why flagged, and the rule or approved fact cited.

**4.** See a verified suggested fix for each finding. Accept, edit or dispute it. Nothing changes without a click.

**5.** Press **Submit for review**. See where it is routed and why (manager or reviewer).

**6.** Switch role (top bar) to Reviewer, open the queue, read comments, and approve or request changes. Download the approval receipt.

**7.** Open Facts, change the offer from 20% to 25%, and see which existing approved items are now wrong (reverse alert).

### UI rules that protect the score

- No signup or password. A role switcher (Creator, Manager, Reviewer, Admin) at the top issues a session token. Permissions are still enforced on the server.
- Pre-seeded data so no screen is ever empty. Every page explains itself in one sentence.
- A visible 'AI use' badge on every report stating which model ran and what text was sent.
- If the LLM is unavailable the app keeps working with the rule-based engine and says so.

## 4. Features and acceptance tests

Priority: **P0** must work for the core path, **P1** strengthens the score, **P2** only after P0 and P1 are done. Each feature has an acceptance test that Claude Code (and you) can run.

| ID | Feature | What it does | Acceptance test | Pri |
|---|---|---|---|---|
| F1 | Brand knowledge base | Rules, approved facts (owner, expiry), glossary, palette, seeded for demo brand Brewloom. View page plus Admin edit | Open Knowledge Base: 12 rules, 7 facts, glossary and palette visible; Admin can edit a fact | P0 |
| F2 | Content intake | Paste text and optionally upload an image; choose channel; 3 example buttons | Clicking an example fills the form and review runs in under 10 s | P0 |
| F3 | Rule-based checks | Terms, brand name, tone, jargon, sentence length, link text, alt text, contrast, claims, facts, expired facts | Example 1 returns all findings listed in Section 9 | P0 |
| F4 | AI judgment checks | Gemini reviews tone and claims using only retrieved rules and facts; NVIDIA fallback | With key set, report shows 'AI used: Gemini'; with key unset, app still works | P1 |
| F5 | Citation validator | Drops or downgrades any AI finding without a real rule/fact ID and real text span | A fake rule ID from the model becomes 'human decision needed', never shown as a rule | P0 |
| F6 | Explained findings | Highlighted span, severity, why, cited source quote, confidence | Every finding card shows source ID and quote, or the 'no rule found' label | P0 |
| F7 | Verified suggestions | Candidate fix, re-check, meaning-preserved score; shown only if it passes | Fixed text produces zero new findings; shows 'intent kept N%' | P0 |
| F8 | Human decisions | Accept (creates new version), edit, dispute with reason | Accepting creates version 2 only after a click; disputes are stored with reason | P0 |
| F9 | Routing and approval | Auto-route to manager or reviewer; server blocks wrong role; approve or request changes | Manager cannot approve Example 3; Reviewer can | P0 |
| F10 | Comments, versions, audit log | Threaded comments per content, version list with diff, immutable audit entries | Every action appears in the audit log with actor, role, time | P0 |
| F11 | Approval receipt | Downloadable JSON/PDF: version, rules checked, sources, AI used, data sent, approver | Receipt downloads for an approved item | P1 |
| F12 | Fact expiry and reverse alerts | Edit a fact or move the demo date; scan existing content; list now-wrong items; alert owner | Change offer to 25%: the seeded approved email appears in alerts | P0 |
| F13 | Insights | Top violated rules, by creator or agency, acceptance vs dispute rate, unclear-rule table with clarity score | Dashboard has data from seeded history, not zeros | P1 |
| F14 | AI disclosure page | Which model, what data is sent, redaction setting, what is stored | Page reachable from the footer on every screen | P1 |

| ID | Feature | What it does | Acceptance test | Pri |
|---|---|---|---|---|
| F15 | Agent API | POST /api/agent/check and GET /api/agent/context for AI tools | curl example in README returns findings JSON | P1 |
| F16 | Image checks | Dominant colours vs palette, alt text required, contrast of supplied text colours | Off-palette image gets a lower-confidence visual finding | P1 |
| F17 | 3D experience | React Three Fiber hero and an insights 'rule constellation' (see Section 12) | Loads lazily; page works with 3D disabled | P2 |
| F18 | Reliability | Loading and error states, request timeouts, AI retry then fallback, rate limiting, input validation, health endpoint, engine tests | Empty text, huge text, wrong file type and a failing AI key all give a clear message, never a crash | P0 |

## 5. Step-by-step product workflow

| # | Who | Step | System behaviour |
|---|---|---|---|
| 1 | Admin | Seeds or edits the knowledge base | Rules, facts, glossary and palette stored in MongoDB. Embeddings created for each rule and fact |
| 2 | Creator | Pastes text, uploads an image, picks channel, presses Review | Content and version 1 saved; review job runs |
| 3 | System | Runs rule-based checks | Terms, tone, claims, fact numbers, expiry, accessibility, palette. Each finding tagged with rule or fact ID |
| 4 | System | Retrieves the most relevant rules and facts for the text | Embedding similarity picks top rules and all valid facts |
| 5 | System | Calls the LLM (if enabled) with only that context | Sensitive strings redacted first. Model returns structured JSON findings |
| 6 | System | Validates every AI finding | Rule ID must exist, span must exist in the text. Otherwise it becomes 'human decision needed' and routes to a reviewer |
| 7 | System | Generates and verifies suggestions | Candidate fix, re-run checks on fixed text, compare meaning. Show only passing fixes |
| 8 | Creator | Accepts, edits or disputes each finding | Accept creates a new version and re-checks. Dispute stores the reason for insights |
| 9 | Creator | Submits for review | Routing: any unsupported or health claim, competitor comparison or low-confidence finding goes to Reviewer; otherwise Manager |
| 10 | Manager / Reviewer | Reads findings and comments, approves or requests changes | Server enforces role. Approval blocked while a high finding is unresolved |
| 11 | System | Writes approval receipt and audit entries | Receipt lists version, sources, AI used, data sent, approver, time |
| 12 | Admin | Changes a fact or the demo date | All non-archived content rescanned; reverse alerts created for items now wrong |
| 13 | Manager / Admin | Opens Insights | Recurring violations, dispute rates, unclear rules, fix acceptance |

### Role permissions

| Action | Creator | Manager | Reviewer | Admin |
|---|---|---|---|---|
| Submit content, decide on findings | Yes | Yes | Yes | Yes |
| Approve low-risk content | No | Yes | Yes | Yes |

| Action | Creator | Manager | Reviewer | Admin |
|---|---|---|---|---|
| Approve content with claim or low-confidence issues | No | No | Yes | No |
| Edit facts, rules, glossary, demo date | No | No | No | Yes |
| View insights | Own only | Yes | Yes | Yes |

# 6. Review engine design

Principle: **use code wherever a rule can be checked exactly, and use the LLM only for judgment.** This keeps results reliable and explainable.

## Check catalogue

| Check | Method | Cites | Severity |
|---|---|---|---|
| Banned or preferred terms (guests, not customers) | Glossary word-boundary match | R-TERM-1 | Medium |
| Brand name spelling (Brewloom) | Case and spacing variants | R-TERM-2 | Medium |
| Hype tone: more than one '!' or ALL-CAPS words | Count and regex | R-TONE-1 | Low |
| Slang or text-speak (OMG, gonna) | Lexicon | R-TONE-2 | Low |
| Jargon and sentences over 25 words | Lexicon and word count | R-TONE-3 | Low |
| Superlatives or absolutes (best, fastest, guaranteed) | Pattern. Supported only if an approved claim fact matches | R-CLAIM-1 or fact ID | High if unsupported |
| Health claims (boosts immunity, weight loss) | Lexicon, always reviewer-routed | R-CLAIM-2 | High |
| Competitor comparison (better than X) | Competitor list | R-MSG-1 | High |
| Offer, price, delivery or city number differs from approved fact | Fact pattern and value compare | Fact ID | High |
| Mention of an expired offer | Fact pattern and valid_until vs demo date | Fact ID | High |
| Non-descriptive link text (click here) | Pattern | R-A11Y-2 | Low |
| Image without alt text | Field check | R-A11Y-1 | Medium |
| Text and background contrast under 4.5:1 | WCAG contrast formula | R-A11Y-3 | Medium |
| Image colours off palette | Dominant colours vs palette distance | R-VIS-1 | Low, labelled lower confidence |
| Tone, claim and messaging judgment beyond the above | LLM with retrieved context, validated | Rule ID or 'no rule found' | Varies |

## Citation validator (the centrepiece)

- LLM output is forced into a JSON schema: `{ruleId, factId, span, issue, confidence, suggestion}`.
- Reject any finding whose rule or fact ID is not in the database, or whose span is not an exact substring of the content.
- Rejected findings are not dropped silently: they become **needs human decision** with confidence 'low' and route to a Reviewer.
- Never display a rule the system did not retrieve from the knowledge base.

## Verify-then-suggest loop

- Generate candidates: rule-based transforms (term swap, remove exclamation, plain-word swap, replace claim with the approved fact statement) plus, if enabled, an LLM rewrite constrained to approved facts.
- Re-run all deterministic checks on the full fixed text. Reject the candidate if the original issue remains or any new issue appears.
- Meaning check: embedding similarity between original and candidate sentence, plus the share of original content words retained. Show it as 'intent kept N%'. Reject below the threshold you set (start at 70%).
- If nothing passes, show 'no verified fix, human rewrite needed'.

### Fact expiry and reverse alerts

- Each fact has: topic, statement, detection pattern, approved value, owner, valid_until.
- A global 'demo date' (default today) lets a judge move time forward and watch facts expire.
- On any fact edit or date change, rescan the latest version of every content item not archived. Create an alert per item whose findings now include that fact.
- Alert shows content title, exact outdated text, the approved replacement, and the owner.

### Guideline clarity score

- For each rule: dispute rate (disputed findings divided by decided findings) from creator and reviewer decisions.
- Clarity = 100 minus the dispute rate as a percentage, shown only after at least 5 decisions. Rules under 70 are listed as 'likely ambiguous, review wording'. Rule test cases and automated edge-case generation belong to the Round 3 testing lab, so do not build them now.
- Describe it honestly as a flag for human review, not as an automatic judgment.

## 7. Technology stack

| Layer | Choice | Why this choice |
|---|---|---|
| Frontend | React 18 + Vite + Tailwind CSS | Fast to build, easy for Claude Code to generate, fast hot reload |
| 3D and animation | three + @react-three/fiber + @react-three/drei, Framer Motion for UI motion | React-friendly 3D. Lazy-loaded so the core path stays fast |
| State and data | TanStack Query + React Router | Simple server-state handling and routing |
| Backend | Node.js 20 + Express (single language with the frontend) | One language end to end, quick to build and deploy |
| Validation | Zod for request and AI-output schemas | The citation validator and API inputs share one schema system |
| Database | MongoDB Atlas (free M0) with Mongoose | Flexible documents suit findings, versions and audit logs. Free tier is enough |
| Retrieval | Gemini embeddings stored in MongoDB; cosine similarity in memory (about 25 documents) | Simple and robust for demo scale. Atlas Vector Search is a Round 3 upgrade |
| LLM | Gemini (Flash-class model, free tier) via Google AI Studio key. NVIDIA NIM (OpenAI-compatible endpoint from build.nvidia.com) as fallback | Free keys, structured JSON output, two providers for reliability |
| Image handling | multer for upload, sharp for resizing and dominant colours | Reliable on Linux hosts; no native-binary surprises on Render |
| Auth | JWT issued by role picker, no password | Judges enter without signing in, server still enforces roles |
| Receipts | pdfkit or JSON download | Light, no browser needed |

| Layer | Choice | Why this choice |
|---|---|---|
| Testing | Vitest for the engine, one Playwright smoke test of the core path | Protects the scored path from regressions |
| Hosting | Render web service (Express serves the built React app) + MongoDB Atlas | One public URL, no CORS problems. See Section 10 |

**Model names change.** Do not hard-code a model ID from this document. Copy the current Gemini model name from Google AI Studio and the NVIDIA model name and base URL from the model page on build.nvidia.com, and put them in environment variables.

# 8. Architecture, data model and API

## Flow

| Browser (React + Three.js) | Express API | Review engine | MongoDB Atlas |
|---|---|---|---|
| Pages, role switcher, highlights, 3D scenes | Routes, JWT role check, validation, audit writer | Rules checks, retrieval, LLM call, citation validator, suggestion verifier | rules, facts, content, versions, findings, comments, audit, alerts |

The engine is a pure module (text in, findings out) so it can be tested without the server and reused by the agent API.

## Collections

| Collection | Key fields |
|---|---|
| rules | ruleId, category, title, text, severity, requiresSource, embedding |
| facts | factId, topic, statement, pattern, value, unit, owner, validFrom, validUntil, kind (fact or claim), embedding |
| terms | preferred, banned[], ruleId |
| assets | palette[hex], fonts[], logoRules |
| contents | title, channel, creator, status (draft, in_review, changes_requested, approved), currentVersion, routedTo |
| versions | contentId, n, body, altText, imagePath, author, createdAt, report{aiUsed, provider, charsSent, redactions} |
| findings | versionId, code, ruleId, factId, severity, span{start, end, text}, why, sourceQuote, confidence, needsHuman, suggestion{text, verified, intentKept}, status (open, accepted, edited, disputed), reason |
| comments | contentId, author, role, body, createdAt |
| auditLogs | contentId, event, actor, role, detail, createdAt (insert only) |
| alerts | factId, contentId, outdatedText, approvedText, owner, status, createdAt |
| settings | demoDate, llmProvider, redactBeforeLlm |

## API endpoints

| Endpoint | Purpose |
|---|---|
| POST /api/session | Choose role and name, receive JWT (no password) |
| GET /api/kb | Rules, facts, terms, palette, demo date |
| PUT /api/facts/:id, POST /api/facts | Admin edits facts; triggers rescan and reverse alerts |
| POST /api/settings/demo-date | Move demo date; triggers rescan |
| POST /api/contents (multipart) | Create content with optional image and run review |
| GET /api/contents, GET /api/contents/:id | Queue and detail (versions, findings, comments, audit) |
| POST /api/findings/:id/decision | accept, edit or dispute with reason; accept creates new version |
| POST /api/contents/:id/submit \| approve \| request-changes | Workflow, with server-side role checks |

| Endpoint | Purpose |
|---|---|
| POST /api/contents/:id/comments | Add comment |
| GET /api/contents/:id/receipt | Approval receipt |
| GET /api/alerts, POST /api/alerts/:id/ack | Reverse alerts |
| GET /api/insights | Insights and clarity scores |
| POST /api/agent/check, GET /api/agent/context | Public API for AI tools |
| GET /api/health | Health check, also used to keep the free host awake |

# 9. Demo data (seed)

Fictional brand: **Brewloom**, an Indian specialty coffee brand. Demo date defaults to today. Seed this on first start if the database is empty.

### Guideline rules

| ID | Category | Rule | Sev |
|---|---|---|---|
| R-TONE-1 | Tone | Warm and calm. At most one exclamation mark per piece. No ALL-CAPS words for emphasis | Low |
| R-TONE-2 | Tone | No slang or text-speak (OMG, lol, gonna, wanna, u) | Low |
| R-TONE-3 | Tone | Plain language. Avoid jargon (synergy, leverage, game-changing, revolutionary). Keep sentences under 25 words | Low |
| R-TERM-1 | Terms | Say 'guests', never 'customers', 'users' or 'clients' | Med |
| R-TERM-2 | Terms | Brand name is written 'Brewloom' (one word, capital B only) | Med |
| R-TERM-3 | Terms | Say 'affordable', never 'cheap' | Low |
| R-CLAIM-1 | Claims | Superlatives and absolutes (best, fastest, #1, guaranteed, 100%) are allowed only when an approved claim supports them | High |
| R-CLAIM-2 | Claims | No health claims (boosts immunity, detox, weight loss, cures). Needs legal review | High |
| R-MSG-1 | Messaging | Never compare Brewloom to a named competitor | High |
| R-FACT-1 | Facts | Offers, prices, delivery promises and city counts must match the approved fact and be within its validity dates | High |
| R-A11Y-1 | Accessibility | Every image needs alt text that describes it (5 to 125 characters, no 'image of') | Med |
| R-A11Y-2 | Accessibility | Link text must be descriptive. Never 'click here' | Low |
| R-A11Y-3 | Accessibility | Text and background contrast at least 4.5:1 | Med |
| R-VIS-1 | Visuals | Use only the brand palette in images and graphics | Low |

### Approved facts and claims

| ID | Statement | Detect | Approved | Valid until | Owner |
|---|---|---|---|---|---|
| F-OFFER | 20% off your first order with code WELCOME20 | 'N% off' | 20 | 31 Dec 2026 | Marketing |
| F-DEL | Delivery in 2 days across 120+ cities | 'N-day delivery' or 'delivery in N days' | 2 | 31 Dec 2026 | Operations |
| F-CITIES | We deliver to 120+ cities | 'N cities' | 120 | 31 Dec 2026 | Operations |
| F-SHIP | Free delivery on orders above Rs 499 | 'free delivery ... above N' | 499 | 31 Dec 2026 | Operations |
| F-SALE | Monsoon Sale: 30% off sitewide | 'Monsoon Sale' | 30 | 30 Sep 2026 (expired) | Marketing |

| ID | Statement | Detect | Approved | Valid until | Owner |
|---|---|---|---|---|---|
| C-ROAST | Beans are roasted fresh every week in Kolkata | claim: 'freshest', 'fresh roasted' | n/a | 31 Dec 2026 | Brand |
| C-SPEED | Delivery in 2 days across 120+ cities | claim: 'fastest', 'quickest delivery' | n/a | 31 Dec 2026 | Operations |

### Glossary, palette and competitors

- **Glossary:** guests (not customers, users, clients); Brewloom (not Brew Loom, BrewLoom, brewloom); affordable (not cheap).
- **Palette:** #3B2A20 espresso, #C8963E caramel, #F6EFE6 cream, #1F5F4A leaf. Contrast checks use these.
- **Competitor names (R-MSG-1):** Starbucks, Blue Tokai, Third Wave Coffee, Chaayos.

### Three example contents and expected outcomes

| Example | Text | Expected result |
|---|---|---|
| 1. Instagram caption (many issues) | OMG our MONSOON SALE is live!!! Get 30% off the BEST coffee in India. Cheap prices for all our customers. Click here to order. Brew Loom delivers in 1 day! | Findings: slang (OMG), ALL CAPS and exclamations, expired fact F-SALE, unsupported superlative 'BEST', 'Cheap' to affordable, 'customers' to guests, 'Click here', brand spelling, delivery 1 day conflicts with F-DEL. Routed to Reviewer (unsupported claim, expired offer) |
| 2. Welcome email (clean) | Welcome to Brewloom. Guests get 20% off their first order with code WELCOME20. Beans are roasted fresh every week in Kolkata, and delivery takes 2 days across 120+ cities. | No high findings. Routed to Manager. Approvable. This is the item used for the reverse alert demo |
| 3. Wellness post (reviewer case) | Our cold brew boosts immunity and helps you lose weight, and it is better than Starbucks. Guaranteed to make you healthier. | R-CLAIM-2 health claims, R-MSG-1 competitor, R-CLAIM-1 'Guaranteed'. Manager cannot approve; Reviewer must |

### Seeded history (so dashboards are not empty)

- Two approved, scheduled items: 'Weekend email: Get 20% off your first order' and 'Website banner: Delivery in 2 days'.
- About 25 past findings with decisions across 3 fake creators (Agency Pixel, Priya (Social), Rahul (Email)), including several disputes on R-TONE-3 and R-CLAIM-1 so the unclear-rules table shows something real.
- Mark seeded history clearly as 'sample data' in the UI. Do not present it as real usage.

## 10. Hosting and deployment

| Part | Service | Cost | Notes |
|---|---|---|---|
| Database | MongoDB Atlas M0 | Free | Allow access from anywhere (0.0.0.0/0) for the free host; use a dedicated DB user with a strong password |
| App (recommended) | Render web service: Express serves the built React app | Free tier | One URL, no CORS. Build command: install and build client, then install server. Start: node server/index.js |
| Alternative | Vercel (frontend) + Render (API) | Free | Needs CORS and two URLs. Use only if you prefer it |
| Keep awake | UptimeRobot or cron-job.org pinging /api/health every 10 minutes | Free | Free Render services sleep when idle. A first visit after sleep can take about a minute |
| Code | Public GitHub repo (krrish2803) | Free | No secrets in the repo. Provide .env.example |

### Environment variables

```
MONGODB_URI=
JWT_SECRET=
LLM_PROVIDER=gemini            # gemini | nvidia | none
GEMINI_API_KEY=
GEMINI_MODEL=                  # copy current name from Google AI Studio
GEMINI_EMBED_MODEL=            # current embedding model name
NVIDIA_API_KEY=
NVIDIA_BASE_URL=               # from build.nvidia.com model page
NVIDIA_MODEL=
REDACT_BEFORE_LLM=true
DEMO_DATE=                     # optional, defaults to today
```

### Pre-submission checklist

- Open the live link in a private window. Do the full seven-step path from Section 3 without signing in anywhere.
- Open the GitHub repo in a private window. Confirm it is public and has no keys.
- Wake the server just before submitting and keep the keep-alive ping on until results are announced.
- Confirm the AI keys have quota. Confirm the app still works with LLM_PROVIDER=none.
- Submit at least a few hours before 10:00 AM IST on 28 Oct.

## 11. Responsible AI and security

| Area | What the product does |
|---|---|
| Human oversight | AI only recommends. Nothing is applied until a user clicks. Approval always needs a person. Low-confidence findings escalate |
| Explainability | Each finding shows the cited rule or fact and a quote of it. No source means an explicit 'human decision needed' |
| AI disclosure | Every report states provider, model, characters sent, and whether redaction ran. Disclosure page explains the same |
| Confidentiality | Only the submitted text and the retrieved rule snippets go to the LLM. Emails, phone numbers and long numbers are redacted before sending when REDACT_BEFORE_LLM is on. Provider keys stay on the server |
| Auditability | Insert-only audit log with rule and fact versions used. Receipt per approval |
| Security basics | Server-side role checks, input validation with Zod, upload type and size limits (images only, under 5 MB), rate limiting on review endpoints, no secrets in the repo |
| Honest limits | Visual checks are less reliable than text checks and are labelled lower confidence. Clarity score is a flag for human review. Demo sessions are not real authentication |

Free-tier LLM services may use submitted prompts under their own terms. Because of this, the demo uses only fictional brand data. Say this on the disclosure page.

## 12. Screens and the 3D plan

| Screen | Contents |
|---|---|
| Home / Review | Banner and 'Try an example', text box, image upload, channel, Review button. After review: highlighted text, findings list, decisions, Submit button |
| Content detail | Versions and diff, findings with decisions, comments, audit log, status and routing, approve and request-changes (role aware), receipt download |
| Queue | All contents with status, risk level and who must act. Filter by role |
| Knowledge Base | Rules, facts (with owner and expiry), glossary, palette. Admin can edit facts and the demo date |
| Alerts | Reverse alerts from changed or expired facts, with outdated text and approved replacement |
| Insights | Top violated rules, by creator, acceptance vs dispute, unclear-rules table, rule constellation (3D) |

| Screen | Contents |
|---|---|
| AI and Privacy | Disclosure page |

### Where 3D is used (and where it is not)

- **Hero scene (Home):** a slowly rotating shield made of floating content cards. During a review, a scan beam sweeps across and cards that fail turn amber or red. Keep it under about 150 KB of assets; use primitives, not heavy models.
- **Rule constellation (Insights):** each rule is a node; size equals violations, colour equals clarity score (red means likely ambiguous). Clicking a node opens that rule's findings.
- Rules for 3D: lazy-load the scenes, respect reduced-motion settings, offer a 2D fallback if WebGL fails, never put 3D between the user and the core path.

**Honest advice:** usability is only 10 of 100 points, while 55 go to the core workflow and technical depth. Judges score the working product, not the animation. Build all P0 features first and add the 3D scenes last. If time is short, ship the hero scene only. A smooth 2D app that works end to end beats a beautiful one that breaks.

## 13. 48-hour build plan

About 57 hours remain at the time of writing (Mon 26 Oct, 01:10 IST). Plan to be feature-complete by Tuesday night.

| When (IST) | Goal | Done when |
|---|---|---|
| Mon 26 Oct, morning (3 to 4 h) | Repo, Express, Mongo connection, seed data (Section 9), deploy a hello-world to Render | Public URL serves the app and /api/health |
| Mon, midday to evening | Rule-based engine and tests (F3, F5, F6). Example 1 returns the expected findings | Vitest passes on all 3 examples |
| Mon night | Review screen with highlights and finding cards (F2, F6). Role switcher | A judge can paste text and see explained findings |
| Tue, morning | Suggestions with verify loop and decisions (F7, F8). Versions and audit (F10) | Accept creates version 2; disputes are stored |
| Tue, midday | Routing, approval, comments, receipt (F9, F10, F11) | Manager blocked on Example 3; Reviewer approves |
| Tue, afternoon | Fact expiry, demo date, reverse alerts (F12) | Changing 20% to 25% flags the seeded email |
| Tue, evening | Gemini layer and NVIDIA fallback, redaction, disclosure page (F4, F14). Insights (F13). Agent API (F15) | App works with and without the key |
| Tue night | Image checks (F16). Deploy and run the full seven-step path in a private window | Core path passes on the live URL |
| Wed 28 Oct, early morning | 3D hero and constellation (F17) if all above is done. Updated PDF, feature list, README | Everything submitted before 10:00 AM |

Cut order if you fall behind: F17 (3D), F16 (images), F13 (insights polish), F15 (agent API). Never cut F3, F5, F6, F7, F8, F9, F12.

## 14. Submission pack (use this for the updated PDF)

### Completed features list

*Copy only the features that actually work on the live link. Replace this note with the final list.*

- Brand knowledge base with rules, approved facts (owner and expiry), glossary and palette
- Text and image review with rule-based checks, each finding citing a rule or approved fact
- AI judgment checks (Gemini, NVIDIA fallback) with citation validator and 'human decision needed' escalation
- Verified suggestions: re-checked fixes with an intent-preserved score

- Accept, edit or dispute decisions with versioning
- Role-based routing and approval, comments, audit log and approval receipt
- Fact expiry with reverse alerts and a movable demo date
- Insights: recurring violations, dispute rates, unclear-rules flags
- AI-use disclosure page and a public API for AI tools

## Technology and implementation details

Use Sections 7 and 8, with the exact model names you ended up using.

## Changes from the original Round 1 plan

| Round 1 plan | What was built | Reason |
|---|---|---|
| Next.js frontend | React + Vite + Three.js | Faster for a solo 48-hour build and enables 3D |
| FastAPI backend | Node.js + Express | One language across the stack |
| Postgres + pgvector | MongoDB Atlas, embeddings stored in documents, similarity computed in memory | Free, simple to host, enough for demo scale. Atlas Vector Search planned later |
| Claude for reasoning | Gemini, with NVIDIA fallback | Free-tier keys available for a live public demo |
| Role-based access | Role-picker sessions with server-side checks, no password | Judges must use the product without signing in |
| MCP server | Plain REST agent API in Round 2 | MCP moved to Round 3 |
| Not in plan | 3D hero and rule constellation | Added to make results easier to understand visually |

*Edit this table so it matches what you really built.*

## Progress against the Round 1 plan (judges will compare)

| Round 1 commitment | Status on the live link | Note |
|---|---|---|
| Text and image review | [Built / Partial] | Fill in honestly |
| Guideline, fact and glossary ingestion | [Built / Partial] | Seeded demo brand plus Admin editing |
| Grounded findings with citations | [Built / Partial] | Citation validator and needsHuman escalation |
| Fact expiry with reverse alerts | [Built / Partial] | Your main differentiator. Make it perfect |
| Verified suggestions | [Built / Partial] | Re-check plus intent-kept score |
| Roles, comments, approvals, version history | [Built / Partial] | Role picker with server-side checks |
| Insights and unclear-guideline flags | [Built / Partial] | Dispute-based clarity in Round 2 |
| API for AI tools | [Built / Partial] | REST now, MCP in Round 3 |
| PDF and slides | Round 3 | Was roadmap in Round 1 |

Judges reward a plan that visibly happened. Do not hide a gap: mark it 'Partial' or 'Round 3' and say why.

## Three major improvements for Round 3

| # | Improvement | Why it is a real addition |
|---|---|---|
| 1 | **MCP server** exposing check, get-rules and get-facts tools, demonstrated with a real AI client that checks its own draft before submitting | Moves governance to the point where AI content is created. New integration surface, not a UI change |
| 2 | **PDF and slide-deck review**: extract text and images, run all checks, map findings to page or slide | Adds two content formats the brief asks for, with new parsing pipeline |
| 3 | **Automated guideline testing lab**: generate edge cases per rule, collect Creator, Manager and Reviewer labels, measure disagreement, propose reworded rules for approval | Upgrades the Round 2 clarity flag into a measured, workflow-backed feature |

Show groundwork in Round 2 without building these: the agent API is the base for the MCP server, the engine takes plain text so PDF extraction can plug in, and every rule dispute is already stored for the testing lab. Mention this in one line each.

# 15. Claude Code prompts and repo structure

Create an empty repo, open it in Claude Code, and paste the master prompt first. Then paste one phase prompt at a time. After each phase, run the tests and commit. Give Claude Code this whole PDF (or its text) as context.

## Repo structure

```
brandguard/
  client/                 React + Vite + Tailwind + R3F
    src/pages/            Review, Content, Queue, KnowledgeBase, Alerts, Insights, Privacy
    src/components/       FindingCard, HighlightedText, RoleSwitcher, Scene3D
  server/
    index.js              Express app, serves client/dist
    routes/               session, kb, contents, findings, workflow, alerts, insights, agent
    engine/               checks.js, retrieval.js, llm.js, validator.js, suggest.js, factscan.js
    models/               Mongoose schemas
    seed/                 rules.json, facts.json, terms.json, examples.json, history.json
    tests/                engine.test.js
  e2e/                    core-path.spec.js (Playwright)
  .env.example  README.md  render.yaml
```

## Master prompt

```
You are building BrandGuard, a pre-publication brand-governance web app for a hackathon (Codex 48,
Round 2).
Read the attached specification fully. Rules for this whole project:
1. The core path must work end to end with no login: content in -> checks against approved
rules/facts/terms/assets -> explained findings with cited sources -> verified suggested fixes ->
human accepts/edits/disputes -> routed approval by role.
2. Stack: React 18 + Vite + Tailwind + react-three-fiber (client); Node 20 + Express + Mongoose + Zod
(server); MongoDB Atlas; Gemini with NVIDIA as fallback; app must also work with LLM_PROVIDER=none.
3. Every finding must cite a real ruleId or factId from the database, or be labelled needsHuman.
Never invent rules.
4. Never apply a suggestion without a user click. Server enforces roles.
5. Seed the demo data exactly as in Section 9 on first start.
6. Keep the engine a pure module (text in, findings out) with Vitest tests for the three examples.
7. Do not put secrets in the repo. Provide .env.example.
8. Build in phases. After each phase, run tests and tell me what works and what does not. Do not start
3D work until I say so.
Start by proposing the file structure and then implement Phase 1.
```

## Phase prompts

### Phase 1: foundation and seed

```
Create the repo structure, Express server that serves client/dist, Mongoose models, MongoDB
connection, and seed scripts for rules, facts, terms, palette, three examples and seeded history from
Section 9. Add GET /api/health, GET /api/kb and POST /api/session (role picker, JWT, no password).
Add render.yaml and a README with run and deploy steps.
```

### Phase 2: rule-based engine

```
Implement server/engine/checks.js with the check catalogue in Section 6 (terms, brand name, tone,
jargon, long sentences, link text, superlatives with approved-claim lookup, health claims, competitor
comparison, fact number mismatch, expired facts using the demo date, alt text, contrast). Each
finding has code, ruleId or factId, severity, span, why, sourceQuote, confidence, needsHuman. Write
Vitest tests so Example 1, 2 and 3 produce the expected outcomes in Section 9.
```

### Phase 3: review UI

```
Build the Review page: banner with Try an example buttons, text area, image upload, channel select,
Review button, highlighted text with severity colours, and finding cards showing why, cited source
quote and confidence. Add the role switcher in the top bar and an AI-use badge. Wire POST
/api/contents and GET /api/contents/:id.
```

### Phase 4: suggestions and decisions

```
Implement verify-then-suggest: generate candidate fixes, re-run all checks on the full fixed text, reject candidates that leave the issue or add new ones, compute intent kept using word overlap now (embedding similarity added in Phase 6). Add accept (creates a new version only on click), edit, and dispute with reason. Add versions list with diff and the audit log.
```

### Phase 5: workflow

```
Implement submit, routing rules (reviewer for unsupported or health claims, competitor comparisons, low confidence or needsHuman; otherwise manager), approve and request-changes with server-side role checks, comments, queue page, and the approval receipt download. Block approval while a high finding is unresolved.
```

### Phase 6: facts, alerts and AI layer

```
Implement Admin fact editing and the demo-date control. On change, rescan latest versions of all non-archived contents and create alerts for items now wrong. Build the Alerts page. Then add the LLM layer: retrieval with embeddings, redaction, Gemini call with a Zod JSON schema, NVIDIA fallback, the citation validator that downgrades invalid findings to needsHuman, and the AI disclosure page. Everything must still work with LLM_PROVIDER=none.
```

### Phase 7: insights, agent API, image checks

```
Build GET /api/insights and the Insights page (top rules, by creator, accept vs dispute, unclear-rules table with clarity score from dispute rate only; do not build rule test cases). Add POST /api/agent/check and GET /api/agent/context with a curl example in the README. Add image checks with sharp: dominant colours vs palette, required alt text, contrast of supplied colours.
```

### Phase 8: test and deploy

```
Write a Playwright test that performs the seven-step core path from Section 3 against a running server. Fix failures. Deploy to Render with Atlas, set env vars, add the keep-alive instructions to the README, and verify in a private window.
```

### Phase 9: 3D (only if everything above passes)

```
Add lazy-loaded react-three-fiber scenes: a hero scene with floating content cards and a scan beam that colours cards by result, and a rule constellation on the Insights page (node size = violations, colour = clarity). Respect prefers-reduced-motion and provide a 2D fallback if WebGL is unavailable. Do not change the core path.
```

## 16. Risks and honest notes

| Risk | What to do |
|---|---|
| Solo build in about 57 hours; scope too big | Follow the cut order in Section 13. P0 features first, 3D last |
| 3D eats the time and breaks the core path | Phase 9 only after the Playwright core-path test passes. Lazy-load and keep a 2D fallback |
| Free Render service sleeps; judge sees a long wait | Keep-alive ping, plus a loading screen that says the server is waking up |
| Gemini or NVIDIA rate limits or an invalid key during judging | Rule-based engine works alone; the report states when AI was unavailable. Test with LLM_PROVIDER=none |
| AI invents rules or spans | Citation validator and needsHuman downgrade (F5). Test it with a fake rule ID |
| Link needs a sign-in or repo is private | Test both in a private window before submitting |
| Claiming features that do not work | List only what works. A judge who finds a false claim will distrust the rest |
| Seeded history looks like real usage | Label it 'sample data' in the UI and in the README |

Round 1 evidence (interviews and survey) is still valuable. If you have time, add a short 'what we learned' paragraph to the updated PDF, but only after the product works. A working core path scores more than extra research in this round.